Natív Programozás Gyakorló

SZTE Szoftverfejlesztés Tanszék

2026. őszi félév

Technikai ismertető

- A programot C++ nyelven kell megírni.
- A megoldást a *Bíró* fogja kiértékelni.
  - A Feladat beadása felületen a Feltöltés gomb megnyomása után ki kell várni, amíg lefut a kiértékelés. **Kiértékelés közben nem szabad az oldalt frissíteni vagy a Feltöltés gombot újból megnyomni** különben feltöltési lehetőség veszik el!
- Feltöltés után a *Bíró* a programot `g++` fordítóval és a
  `-std=c++20 -Wall -Werror -static -O2 -DTEST_BIRO=1`
  paraméterezéssel fordítja és különböző tesztesetekre futtatja.
- A program működése akkor helyes, ha a tesztesetek futása nem tart tovább 5 másodpercnél és hiba nélkül (0 hibakóddal) fejeződik be, valamint a program működése a feladatkiírásnak megfelelő.
- A *Bíró* által a `riport.txt`-ben visszaadott lehetséges hibakódok:
  - Futási hiba 6: Memória- vagy időkorlát túllépés.
  - Futási hiba 8: Lebegőpontos hiba, például nullával való osztás.
  - Futási hiba 11: Memória-hozzáférési probléma, pl. tömb-túlindexelés, null pointer használat.
- A beadások számát és a beadási határidőt a bíróban lehet megtekinteni.
- A programban a `#ifndef TEST_BIRO` és a hozzá tartozó `#endif` közötti rész nem lesz figyelembe véve a kiértékelés során.
- A bíróba egyetlen `.cpp` forrásfájlt lehet feltölteni (zip fájlt nem fogja kiértékelni).
- Technikai hiba esetén a technikai hiba elhárulása után a hallgató folytathatja a ZH-t, és a ZH határidejét a hibának megfelelően módosítjuk. Ha a hallgató a technikai hiba miatt nem képes folytatni a ZH-t, akkor az adott alkalom igazolt hiányzásnak minősül.
- A ZH megírásához hivatalos fényképes igazolvány szükséges. Ennek hiányában a ZH nem teljesíthető.
- Fordítási hiba vagy figyelmeztetés esetén a beadás 0 pontot ér.
- A feladat csak terminálban oldható meg, IDE nem használható.

Általános követelmények, tudnivalók

- A programnak követnie kell az objektum-orientált programozás elveit!
- Csak a leírásban szereplő osztályokat, metódusokat és adattagokat kell megvalósítani, egyéb dolgokért nem jár plusz pont.
- Minden metódus, amelyik nem változtatja meg az objektumot, legyen konstans! Ha a paramétert nem változtatja a metódus, akkor a paraméter legyen konstans!
- string összehasonlításoknál az egyezés a pontos egyezést jelenti, azaz ha kis-nagy betűben térnek el, akkor már nem tekinthetők egyenlőnek (pl. a "piros" != "Piros")
- A leírásokban bemutatott példákban a stringek köré rakott idézőjelek nem részei az elvárt kimenetnek, azok csak a string határait jelölik. Például ha az szerepel, hogy a példa bemenetre az elvárt kimenet az, hogy `"3 alma"`, akkor az elvárt kimenet idézőjelek nélkül az `3 alma`, de a szóköz szükséges!
  - A tesztesetekben nem lesz ékezetes szöveg kiíratása.
- Az elvárt kimeneteknek karakterről karakterre olyan formátumúnak kell lennie, ami a feladatban le van írva (szóközöket és sortöréseket is beleértve).
- Ha az objektum másolása nem triviális (azaz a fordító által generált másolás nem elegendő), akkor a megfelelő másolást is meg kell valósítani.
- Minden megoldást a `solution` namespace-ben kell megírni (lásd a kiadott feladatokat)!

Kiindulási projekt, megoldás feltöltése

- **A megoldáshoz az előre kiadott osztályok módosítása szükséges lehet.**
  - Nem minden ZH esetében van kiindulási projekt.
- Feltöltéskor ezeket az osztályokat is fel kell tölteni és a módosításokat is pontozhatja a bíró!
- Egyes tesztesetekben a bíró módosított osztályt is használhat ezen kiinduló osztályok helyett, ezzel tesztelve a valóban helyes működést!

Zh alatt használható segédanyag

- **A ZH során használható segédanyag elérhető bíróban.**
  - https://biro.inf.u-szeged.hu/kozos/native/

1. Kiadott fájl

Az előre kiadott fájlba érdemes dolgozni! Itt találhatóak lokális teszt esetek is, mindenképpen érdemes ezeket megvizsgálni! Milyen paramétereket kapnak, miket adhatnak vissza, stb..

2. Feladatok

2.1. Task osztály

Az osztály egy gyakorlófeladatot reprezentál. Tárolja az azonosítóját (id, string), maximális pontszámát (maximum, unsigned) és elért pontszámát (points, unsigned).

Task konstruktor, getterek

A konstruktor a fenti sorrendben kapja az adatokat, az elért pontszám alapértéke 0. Az azonosító nem lehet üres, a maximum 1-100 közé essen, és a pontszám nem lehet nagyobb a maximumnál. Hibás adat esetén dobjon std::invalid_argument kivételt "Invalid task" üzenettel. Legyen minden adathoz konstans getter: getId, getMax, getPoints.

percentage

A feladat eredményét adja vissza egész százalékként. Az osztás maradékát el kell hagyni.

prefix++, postfix++, prefix--, postfix--

A ++ növelje, a -- csökkentse az elért pontszámot eggyel. A pontszám mindig 0 és a maximum között maradjon. A visszatérés kövesse a prefix és postfix operátorok szokásos szemantikáját.

+=, -=, + operátor

A kapott unsigned értékkel növelje, illetve csökkentse a pontszámot, a határok betartásával. A += és -= legyen láncolható. A + új példányt adjon vissza, az eredetit ne módosítsa.

==, !=, < operátor

Két feladat akkor azonos, ha az azonosítójuk megegyezik. A != az == tagadására épüljön. A < szerint a nagyobb egész százalékos eredményű feladat kerül előre, holtversenyben az azonosító lexikografikus sorrendje döntsön.

string konverzió, << stream operátor

A feladatot lehessen explicit módon stringgé alakítani és ostream-re írni. Mindkét esetben az alábbi formátumot használja, záró sortörés nélkül. A stream operátor legyen láncolható.

```
Task[<id>; <points>/<maximum>]
```

2.2. TaskBuilder osztály

Az osztály feladatokat készít. A konstruktor egy azonosítókezdő stringet kap. A newTask egy maximumot kap, alapértéke 10. Adjon vissza egy új, 0 pontos feladatot, amelynek azonosítója "<kezdet>-<sorszám>" alakú. A sorszám 1-től indul, és minden sikeres létrehozás után nő. A getCount a sikeresen létrehozott feladatok számát adja vissza. Hibás maximum esetén a Task konstruktorának megfelelő kivételt dobja.

2.3. Workshop osztály

Az osztály témanevekhez (string) feladatokat tárol. Egy témához egy feladat tartozhat, és egy feladatazonosító legfeljebb egyszer szerepelhet. A konstruktor a műhely nevét kapja, alapértéke "Gyakorlas". Kezdetben a műhely üres.

add, count, unsigned konverzió

Az add egy témanevet és egy feladatot kap, ilyen sorrendben. Ha sem a téma, sem a feladatazonosító nem foglalt, tároljon másolatot és térjen vissza true-val. Egyébként false az eredmény. A count és az explicit unsigned konverzió a tárolt feladatok számát adja vissza.

[] indexelő, locateById

Az indexelő a témához tartozó feladatot adja vissza módosítható referenciaként, konstans műhelyen konstans referenciaként. Hiányzó témánál dobjon std::out_of_range kivételt "No topic" üzenettel. A locateById a kapott feladatazonosítóhoz tartozó témanevet adja vissza, hiány esetén "[UNKNOWN]" az eredmény.

- operátor, erasePrefix

Egy témanevet kivonva a műhelyből törlődjön az adott bejegyzés. A visszatérés true, ha történt törlés, különben false. Az erasePrefix a kapott feladatazonosító-kezdettel rendelkező bejegyzéseket törölje, és adja vissza a törölt mennyiséget. Üres prefix mindenre illeszkedik.

~ és ! operátor

A ~ a < szerinti első feladatot adja vissza konstans referenciaként. A ! a feladatok egész százalékos eredményeinek átlagát adja vissza unsigned értékként, a maradék elhagyásával. Üres műhelynél mindkettő dobjon std::out_of_range kivételt "No task" üzenettel.

>= operátor

Egy unsigned százalékhatárt kapjon, és a legalább ezt elérő feladatok másolatait adja vissza vektorban, a feladatok < szerinti sorrendjében.

() függvényhívás operátor

Egy azonosítókezdetet és egy unsigned százalékhatárt kap. Adjon vissza azonos nevű új műhelyt azokkal a feladatokkal, amelyek azonosítója illeszkedik a kezdetre és eredményük eléri a határt. A témanevek maradjanak meg.

write, << stream operátor

A műhely adatait a kapott ostream-re írja, témanevek szerint növekvő sorrendben. Minden sort sortörés zárjon. Üres műhelynél csak a fejléc jelenjen meg. A << legyen láncolható.

```
Workshop: <name>
<topic>: <task>
...
```